



Universität
Marburg



Sitzung 10: Machine Learning - Overfitting & Generalisierbarkeit

Dr. Philipp Bierwirth

Differentielle Psychologie und Persönlichkeitsforschung

SoSe 2026



Universität
Marburg



Lerntagebuchabgabe

15.09.2026



Inhalt

- Rückblick
- Wie lernen Algorithmen
- Overfitting und Generalisierbarkeit
- Train-Test-Split und Kreuzvalidierung
- Praktische Übung



Universität
Marburg



1. Rückblick

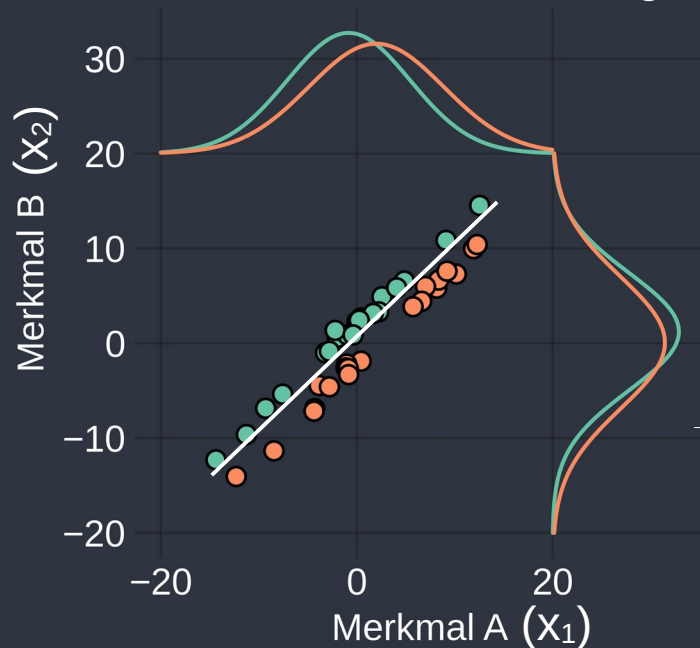


Kombination von Merkmalen

Linearer Klassifikator findet Ebene (im 2d-Fall eine Linie), welche die Gruppen optimal trennt.
z.B.:

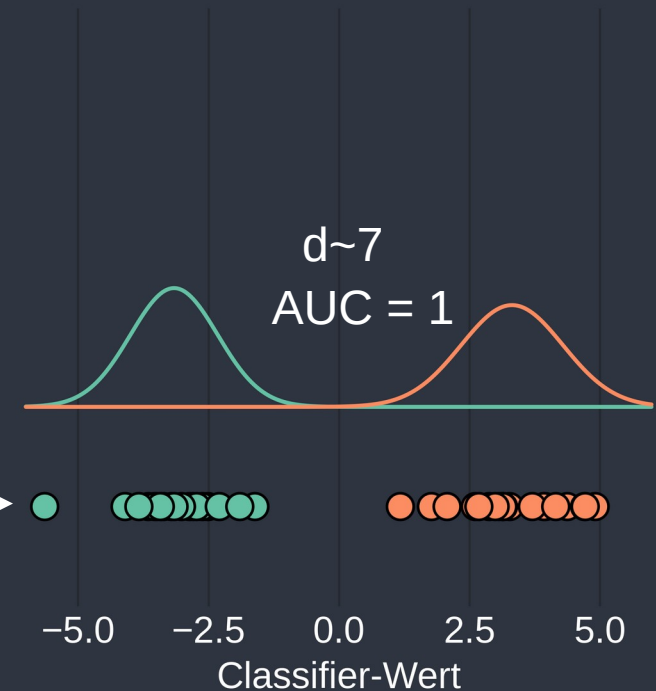
- Logistische Regression
- Support Vektor Maschine
- Neuronales Netzwerk
- Lineare Diskriminanzanalyse
- etc.

Multivariate Betrachtung



$$\beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2$$

- x = Merkmale
- β = Classifier-Gewichte





Universität
Marburg

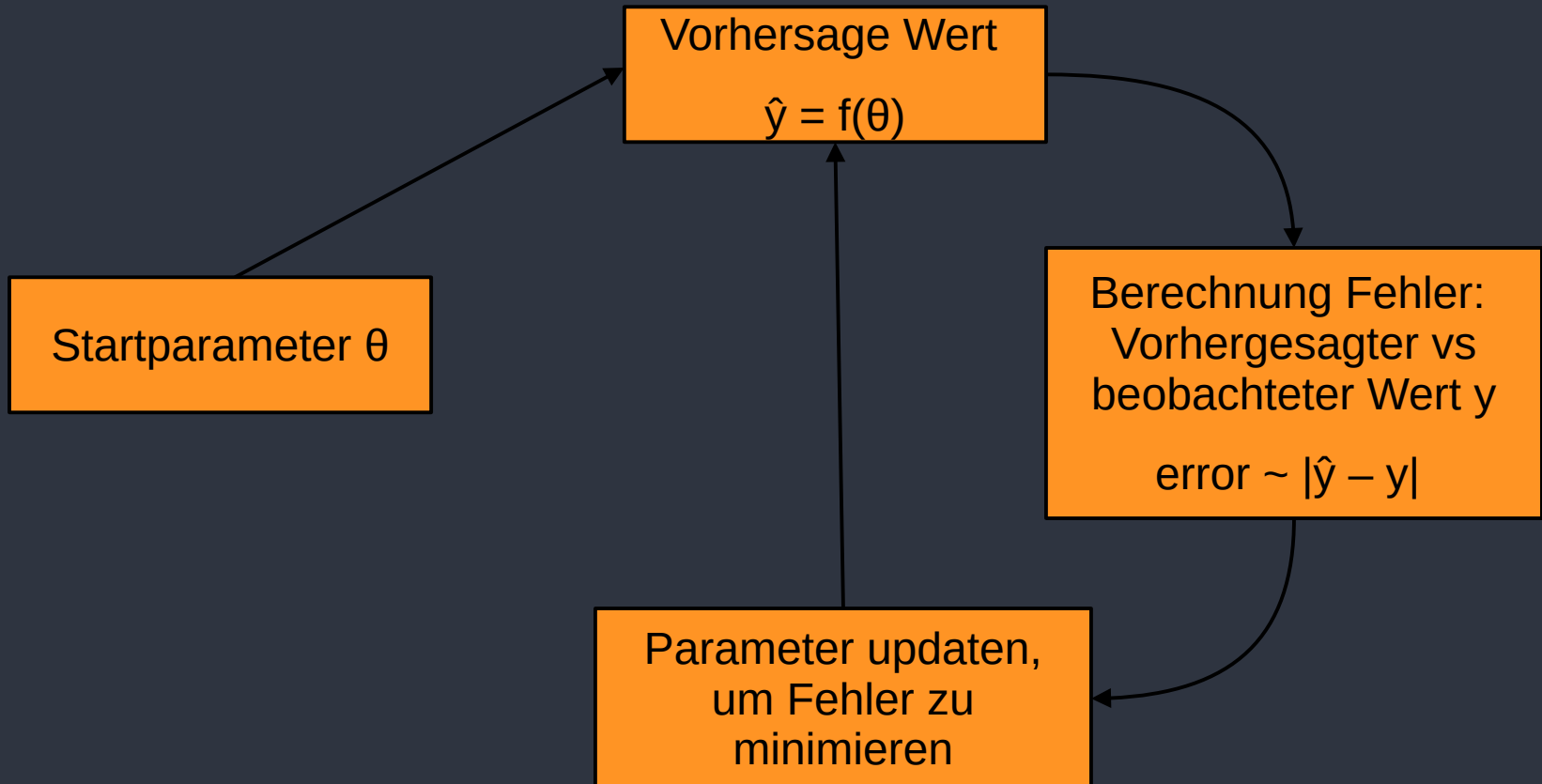


2. Wie lernen Algorithmen



Wie lernen Algorithmen?

- Konzeptionelle Darstellung wie “Maschinen” lernen



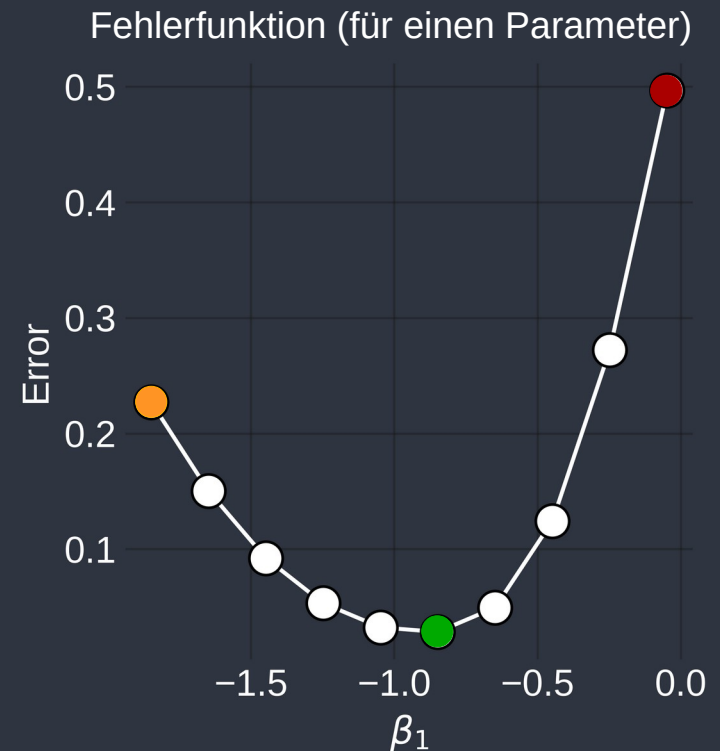
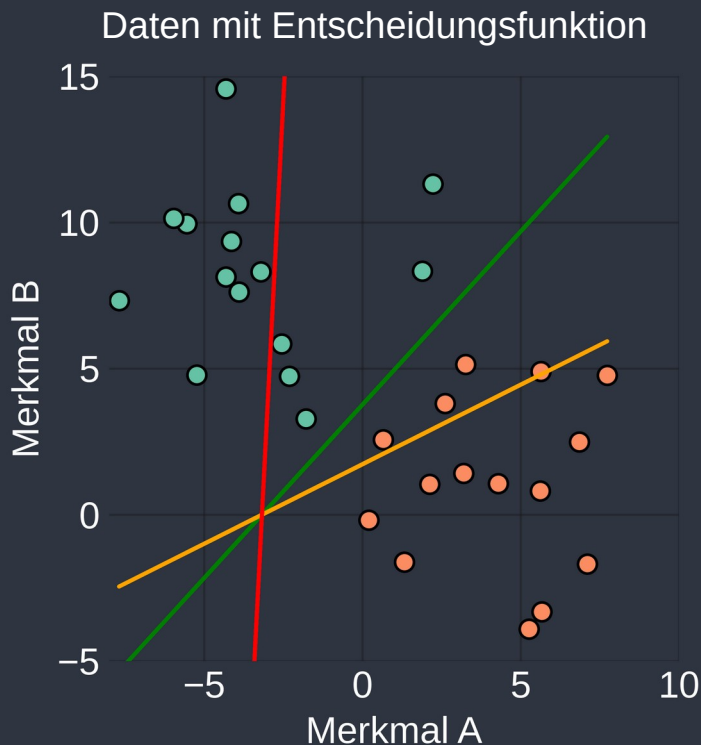


Wie lernen Algorithmen?

- Algorithmen lernen die Parameter, welche die geringsten Fehler/Kosten verursachen
- Wanderung entlang der Fehlerfunktion/Kostenfunktion, um Tal zu finden (geringster Fehler für Parameter)

Beispiel Lineares Modell (z.B.
Logistische Regression):

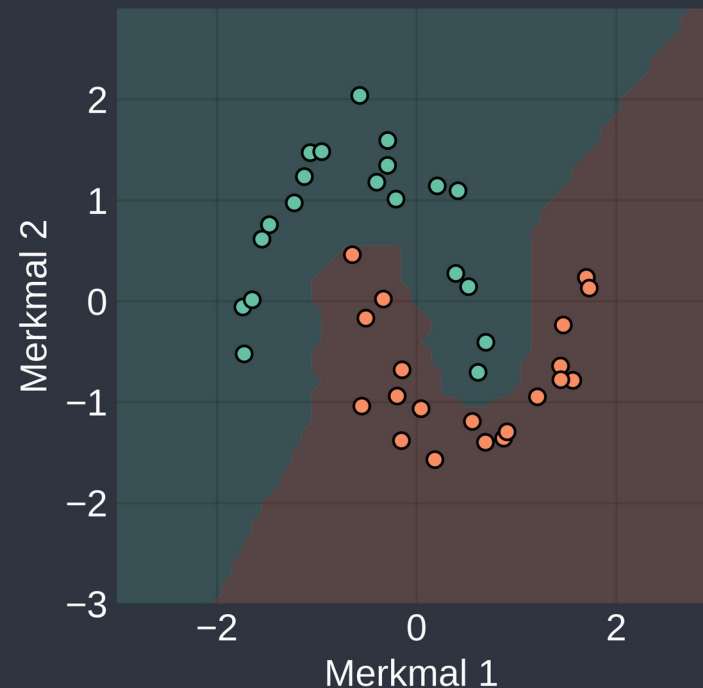
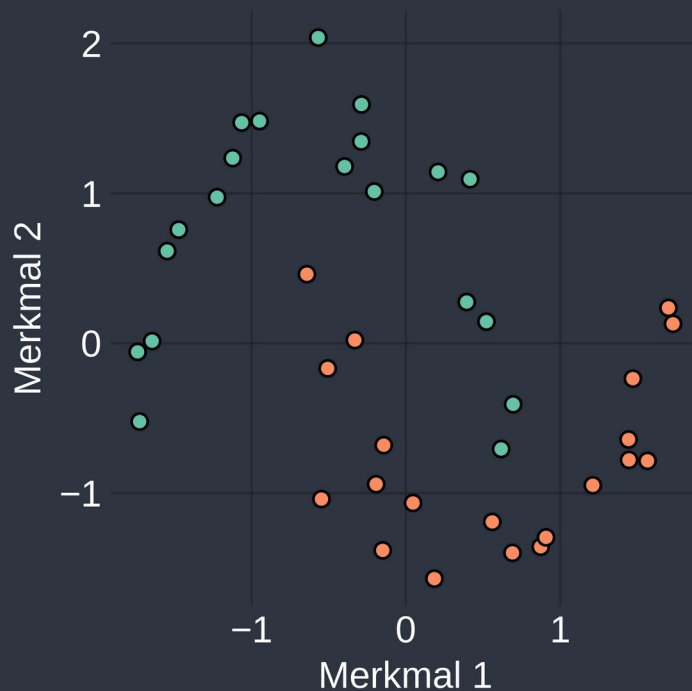
$$\beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2$$





Wie lernen Algorithmen?

- Bestimmte Algorithmen können auch nicht-lineare Entscheidungsfunktionen verwenden, um Daten zu klassifizieren
- z.B. Support-Vektor-Maschinen (mit RBF-Kernel), neuronale Netzwerke, k-Nearest-Neighbor etc.





Universität
Marburg



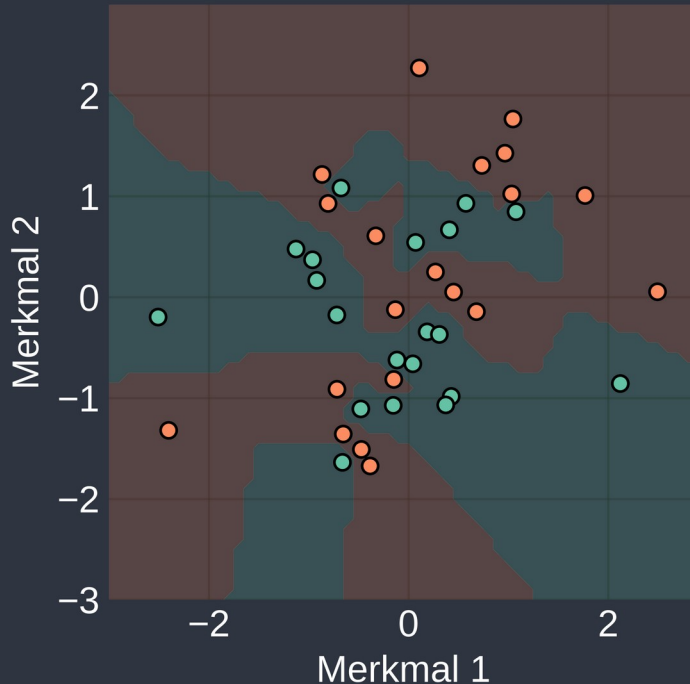
3. Overfitting & Generalisierbarkeit



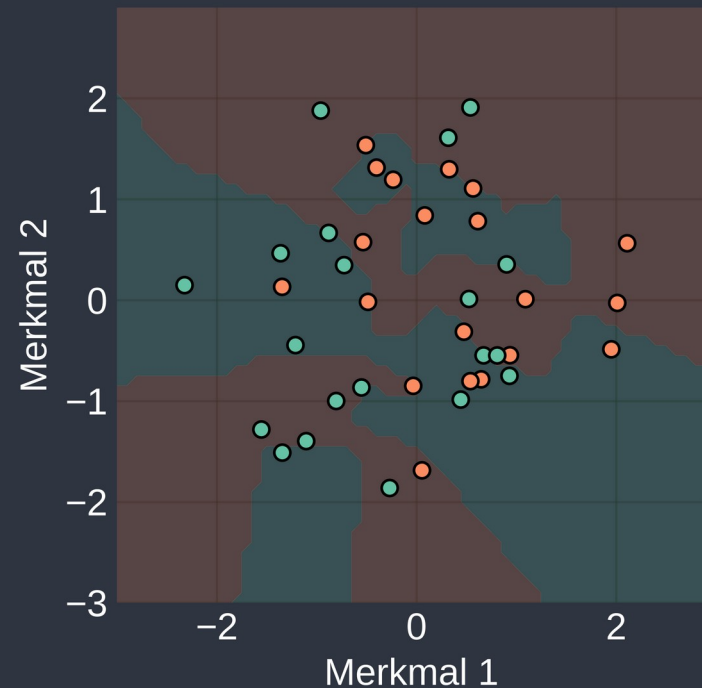
Overfitting und Generalisierbarkeit

- Mehr Flexibilität in den Modellen, führt aber auch dazu, dass evtl. zufälliges Rauschen in den Daten gelernt wird. Das Modell “overfitted” und generalisiert dadurch nicht mehr auf Daten/Beobachtungen, die das Modell noch nicht kennt.
- Beispiel: Ein hochflexibler k-Nearest-Neighbor Algorithmus lernt Modell (links), das alle Beobachtungen korrekt klassifiziert. Wird das Modell mit neuen Daten konfrontiert, fällt die Leistung auf Ratewahrscheinlichkeit zurück (rechts)

Daten, mit denen gelernt wurde



Neue, unbekannte Daten





Universität
Marburg

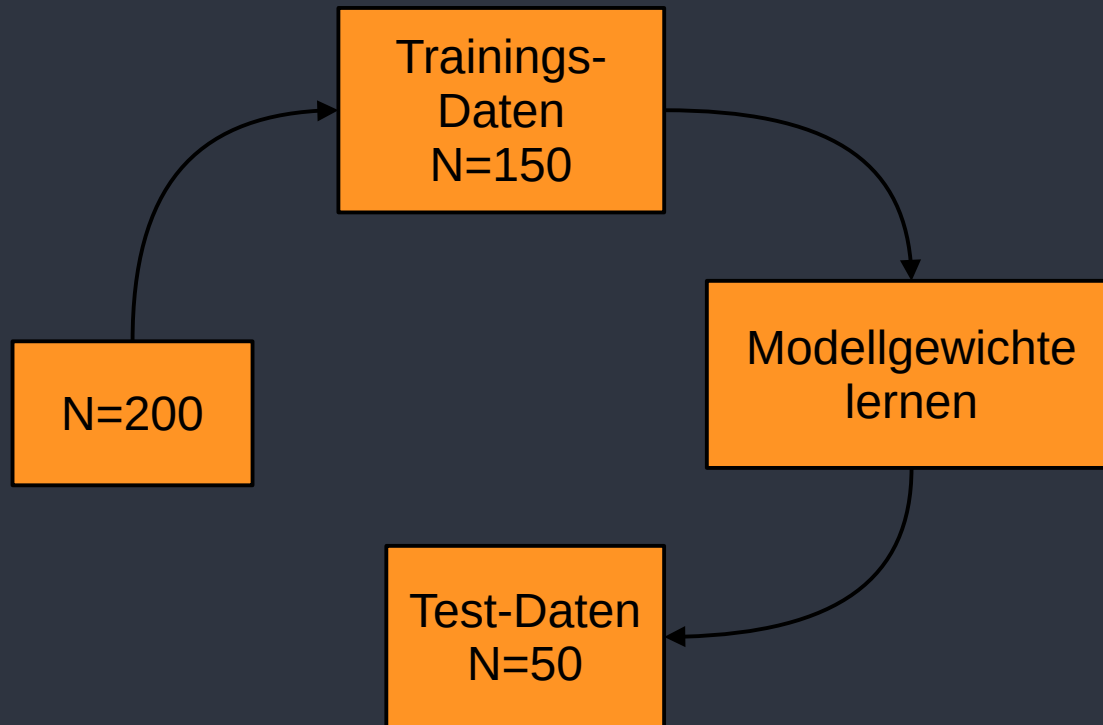


4. Train-Test-Split und Kreuzvalidierung



Train-Test-Split und Kreuzvalidierung

Daten werden in Trainings- und Testdaten gesplitted



Train-Test-Split wichtig, um Overfitting zu vermeiden und Generalisierbarkeit des Modells zu überprüfen

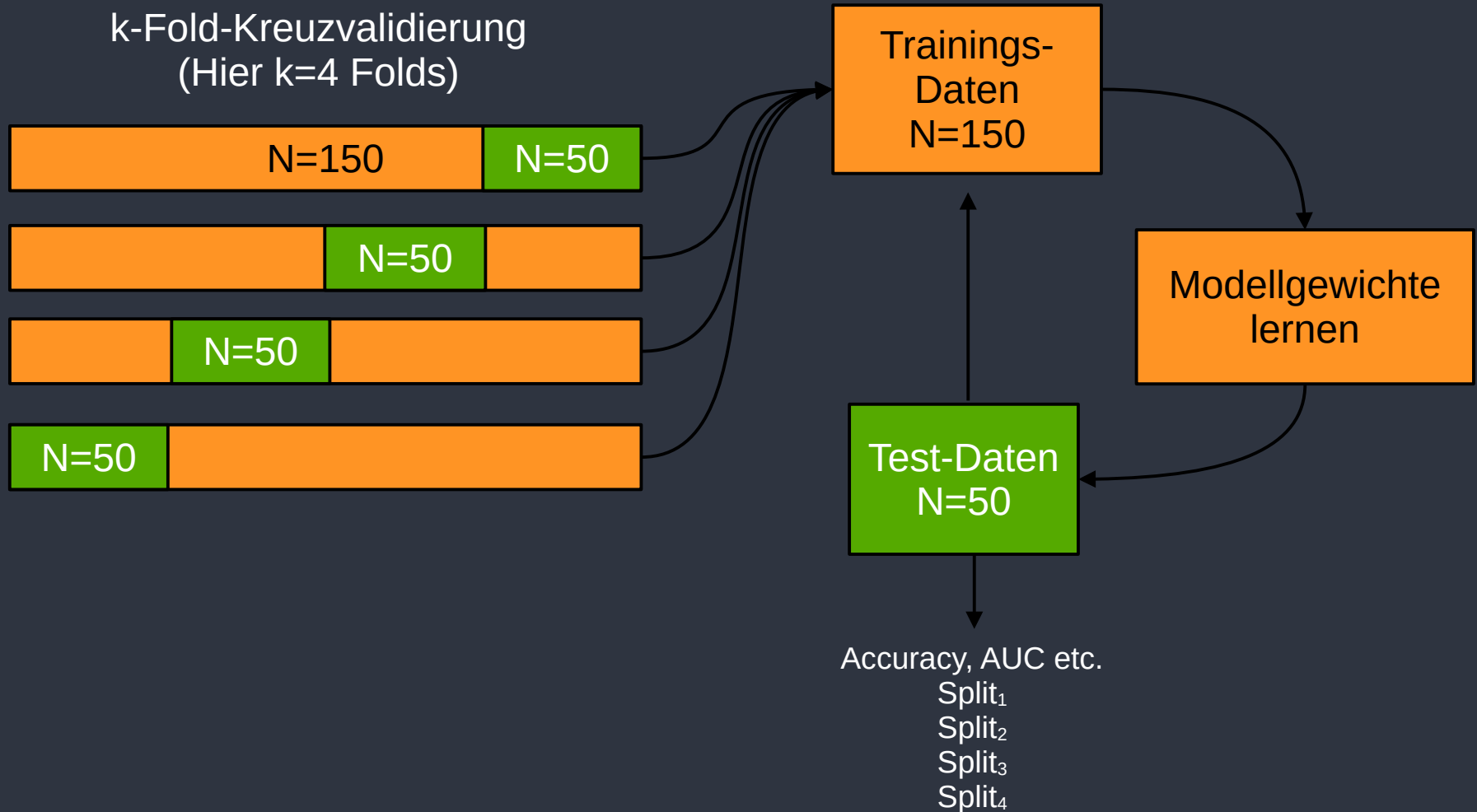
Modellgewichte werden an Trainingsdaten optimiert, indem Fehler minimiert werden

Modellgewichte werden an Testdaten, die das Modell bislang noch nicht gesehen hat, validiert (Berechnung der Akkuratheit, AUC etc.)



Train-Test-Split und Kreuzvalidierung

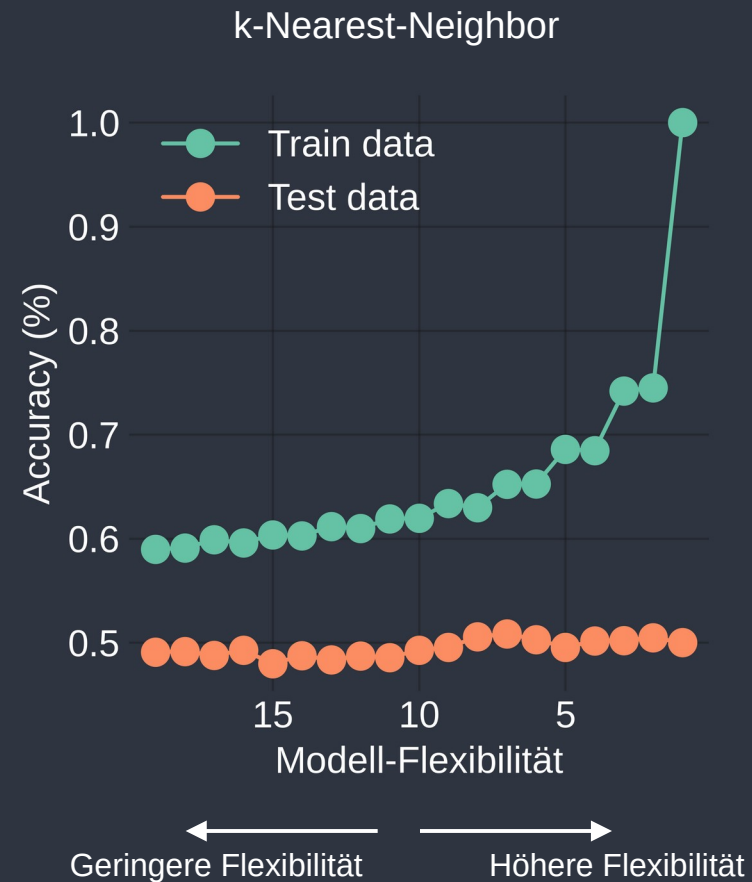
k-Fold-Kreuzvalidierung
(Hier k=4 Folds)





Train-Test-Split und Kreuzvalidierung

- ML-Modelle können - v.a. wenn sie sehr flexibel sind - zufälliges Rauschen verwenden, um Beobachtungen zu klassifizieren
- Substanzielle Diskrepanz zwischen Trainings- und Test-Leistung deutet auf Overfitting hin
- Genauigkeit (Accuracy) sollte für Test-Daten über Ratewahrscheinlichkeit liegen





Universität
Marburg



5. Praxis



Machine Learning in JASP

Machine Learning wird typischerweise in Python durchgeführt (siehe sklearn package) Für den Einstieg bietet JASP aber ein gutes Machine Learning Modul an.

- 1) Laden Sie die Datei "beispieldaten_herzerkrankung.tsv" in JASP.
- 2) Berechnen Sie zunächst t-Tests für unabhängige Stichproben zwischen den beiden Target-Gruppen (0=krank, 1=gesund).
 - Gibt es Unterschiede zwischen den Gruppen? Wie groß sind die Effekte?
- 3) Verwenden Sie einen Algorithmus (z.B. logistische Regression), um die Targetvariable anhand der Merkmalsvariablen vorherzusagen.
 - Stellen Sie einen 'Data Split' von z.B. 20% Testdaten ein.
 - Berechnen Sie die Accuracy und die AUC
 - Visualisieren Sie die ROC
 - Überprüfen Sie, ob die Gruppen gleich groß sind